Name → Krishna Anand

Roll → 23010 1082.

Github → https://github.com/KANAND6031/cloud-computing-Mini-Project

→ What I learnt from this project

From task 1 to 7 taught me the difference between getting a Python script to run on local host and keeping a containerized service in Kubernetes.
I learnt how sequence level distillation compresses a 3.8 B teacher into 70 M student, why model weights must be decoupled from container images into S3 object storages to keep images lightweight.

→ Challenges faced

1) East API Event-loop starvation : Locally, everything worked fine for single curl tests. But once deployed across 2 replicas in K3d under locust load, both pods started failing liveness checks.

Fix → Offloaded CPU ~~inde~~ inference to worker threads

2) During fine tuning the student model kept producing broken JSON missing its opening brace.

Fix → Tokenized the prompt & JSON O/P seperately in raw token ID array.

→ Key Observation & findings

1) Small Model learns grammar for easier than semantics. Evaluating the model revealed an split. Untrained student produced 0% valid JSON. After 3 epoches of CPU training loss fell from 0.69 to 0.08, and JSON validity jumped to 98%.

2) Traefik connection during Rolling updates.

During rolling updates ($V_1 \rightarrow V_2$), terminating old pods immediately caused active locust users to hit dropped connections.

3) When stress testing 50 sequential requests in a container capped at 1 CPU core and 1GB RAM I expected memory to be our tightest constraint. Instead, peak RAM stabilized at just 348.10 MiB. Meanwhile, CPU usage stayed pinned at 100%.

→ What could have been Done better

1) Dynamic Task queue instead of static sharding

2) Semantic checks in the CI quality Gate.

3) Model quantization for CPU speed.

→ Final Takeaway.

Building this pipeline end-to-end from seaweedFS S3 buckets and K3d cluster to CI/CD quality gates proved that working on my laptop is only 20% of the job. Real cloud reliability lives in the details around edge failures connection draining hooks during rollouts, resource cgroup limits and deterministic automation.